การพัฒนาระบบสารสนเทศ อย่างมั่นคงปลอดภัยด้วย AI

Secure Information Systems Development with AI-Powered Tools

Workshop | บรรยาย + ปฏิบัติ | GitHub Copilot + Next.js

นายจีรศักดิ์ สายนาค

นักวิชาการคอมพิวเตอร์

สำนักนวัตกรรมดิจิทัลและระบบอัจฉริยะ มหาวิทยาลัยสงขลานครินทร์

AI

Security

Dev

Agenda

01	**เล่าประสบการณ์ Web Developer** เส้นทาง ปัญหาที่เจอ บทเรียนด้าน Security	บรรยาย
02	**Fundamentals of Web Development** HTML, CSS, JS, HTTP, Client-Server	บรรยาย
03	**Web Security Essentials** OWASP Top 10, XSS, SQL Injection, CSRF	บรรยาย + Workshop
04	**Modern Web Dev with Next.js** React, Components, Routing, API Routes	บรรยาย + Workshop
05	**AI-Powered Development** GitHub Copilot Prompting Strategies	Workshop
06	**Build & Deploy** สร้างเว็บแอปจริง + Deploy บน GitHub Pages	Workshop

01

ประสบการณ์ Web Developer

เส้นทางอาชีพ บทเรียน และความท้าทายด้าน Security

เส้นทาง Web Developer

เริ่มต้น

เรียนรู้ HTML/CSS/JS

สร้างเว็บไซต์แรก เรียนรู้ผ่าน trial & error

เติบโต

Backend & Database

PHP, Node.js, MySQL, MongoDB

มืออาชีพ

Framework & Architecture

React, Next.js, REST API, CI/CD

ปัจจุบัน

AI-Assisted Development

GitHub Copilot, ChatGPT, Secure by Design

บทเรียนด้าน Security จากประสบการณ์จริง

SQL Injection ที่เจอครั้งแรก

เว็บโดน hack จากช่องโหว่ SQL Injection ที่ไม่เคยรู้จักมาก่อน

XSS จาก Comment System

ผู้ใช้แทรก script ผ่านช่อง comment ที่ไม่ได้ทำ sanitization

Password เก็บเป็น Plain Text

ฐานข้อมูลหลุด ข้อมูลผู้ใช้ทั้งหมด ถูกเปิดเผย

API Key หลุดใน GitHub

ลืมใส่ .gitignore ทำให้ API key ถูกเผยแพร่สาธารณะ

“Security ไม่ใช่สิ่งที่เพิ่มทีหลัง มันต้อง built-in ตั้งแต่ต้น”

— บทเรียนสำคัญที่สุดจากประสบการณ์ทำงาน

02

Web Development Fundamentals

พื้นฐานที่ต้องรู้ก่อนพูดถึง Security

How the Web Works

Client (Browser)

HTML
โครงสร้างหน้าเว็บ
CSS
ความสวยงาม ธีม
JavaScript
ฟังก์ชันการทำงาน

HTTP Request →
← HTTP Response

Server (Backend)

API Routes
จัดการ request/response
Database
เก็บข้อมูล
Authentication
ยืนยันตัวตน

Frontend Technologies

HTML

Semantic markup
Accessibility
SEO-friendly structure

CSS

Flexbox & Grid
Responsive Design
CSS Variables

JavaScript

DOM Manipulation
Async/Await
ES6+ Features

HTTP & Security พื้นฐาน

HTTP Methods

GET	ดึงข้อมูล (อ่านอย่างเดียว)
POST	สร้างข้อมูลใหม่
PUT/PATCH	แก้ไขข้อมูล
DELETE	ลบข้อมูล

HTTPS = HTTP + TLS

เข้ารหัสข้อมูลระหว่างทาง
ป้องกันการดักฟัง (MITM)
ยืนยันตัวตนของเซิร์ฟเวอร์
จำเป็นสำหรับทุกเว็บไซต์

03

Web Security Essentials

OWASP Top 10 และช่องโหว่ที่ต้องรู้

OWASP Top 10 — 2021

ช่องโหว่ด้านความปลอดภัยที่พบบ่อยที่สุดในเว็บแอปพลิเคชัน

A01 — Broken Access Control

A02 — Cryptographic Failures

A03 — Injection

A04 — Insecure Design

A05 — Security Misconfiguration

A06 — Vulnerable Components

A07 — Auth Failures

A08 — Data Integrity Failures

A09 — Logging Failures

A10 — SSRF

เราจะเน้น A01, A03, A07 ในเวิร์คชอปนี้

SQL Injection

VULNERABLE

```
query = f"SELECT * FROM users WHERE id = {user_input}"
cursor.execute(query)
```

SECURE

```
cursor.execute(
  "SELECT * FROM users WHERE id = %s", (user_input,)
)
```

ใช้ Parameterized Queries เสมอ — ห้ามใช้ string concatenation

Cross-Site Scripting (XSS)

Reflected XSS

Script ใน URL ถูกสะท้อน
กลับมาในหน้าเว็บ

Stored XSS

Script ถูกเก็บในฐานข้อมูล
แสดงผลให้ผู้ใช้อื่น

DOM-based XSS

JavaScript อ่าน input
แล้วเขียนลง DOM โดยไม่ escape

การป้องกัน

Output Encoding — escape ข้อมูลก่อนแสดงผลเสมอ
Content Security Policy — จำกัด source ของ script
DOMPurify — sanitize HTML ที่ต้องรับจากผู้ใช้

Authentication & CSRF

Password Storage

MD5 / SHA-1 / Plain Text	NEVER
bcrypt / Argon2id	ALWAYS

```
bcrypt.hash(password, 12)
```

CSRF Protection

CSRF Token
สร้าง token สุ่มต่อ session
ส่งพร้อม form submission

SameSite Cookie
ตั้ง SameSite=Lax
ป้องกัน cross-origin request

Custom Headers
SPA ที่ใช้ Bearer token
ปลอดภัยจาก CSRF โดยธรรมชาติ

WORKSHOP

Workshop 1: Security Code Review

สิ่งที่จะทำ

1. Review โค้ดตัวอย่างที่มีช่องโหว่
2. ระบุประเภทช่องโหว่ (SQLi, XSS, CSRF)
3. เขียน fix พร้อมอธิบาย

Copilot Prompt

"Review this code for security vulnerabilities.
Identify each issue, explain the risk, and suggest a fix."

04

Modern Web Dev with Next.js

React, Components, Routing, SSR/SSG

ทำไมต้อง Next.js?

Server-Side Rendering

SEO ดี, โหลดเร็ว

File-based Routing

สร้างไฟล์ = สร้าง route

API Routes

Backend ในตัว
ไม่ต้องแยก server

Static Export

Deploy ได้ทุกที่
รวมถึง GitHub Pages

React Ecosystem

Component-based
reuse ได้ง่าย

Built-in Security

Auto XSS protection
CSP, Headers

Next.js Project Structure

File Structure (App Router)

```
my-app/
├── app/
│   ├── layout.tsx
│   ├── page.tsx
│   ├── about/
│   │   └── page.tsx
│   └── api/
│       └── hello/
│           └── route.ts
├── public/
├── next.config.js
└── package.json
```

สิ่งที่ต้องรู้

`page.tsx` = หน้าเว็บ

`layout.tsx` = โครงร่างหน้า

`route.ts` = API endpoint

โฟลเดอร์ = URL path

```
app/about/page.tsx
→ /about
```

```
app/api/hello/route.ts
→ /api/hello
```

React Components 101

```
export default function Card({ title, description }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      <p>{description}</p>
    </div>
  );
}

// Usage:
<Card title="สวัสดี" description="นี่คือ component" />
```

WORKSHOP

Workshop 2: สร้าง Next.js App แรก

Step 1

```
npx create-next-app@latest my-secure-app
```

สร้างโปรเจกต์ใหม่

Step 2

```
cd my-secure-app && npm run dev
```

เปิด development server

Step 3

แก้ไข `app/page.tsx`

สร้างหน้า Home ด้วย Copilot

Step 4

สร้าง `app/about/page.tsx`

เพิ่มหน้า About

WORKSHOP

Copilot Prompts สำหรับ Workshop 2

สร้าง Layout

```
"Create a Next.js root layout with a navigation bar containing links to Home and About pages. Use
Tailwind CSS."
```

สร้าง Home Page

```
"Create a responsive hero section for the home page with a title, description, and call-to-action
button using Tailwind CSS."
```

สร้าง About Page

```
"Create an About page component with team member cards in a grid layout. Each card has name, role, and
description."
```

05

AI-Powered Development

ใช้ GitHub Copilot อย่างมีประสิทธิภาพ

GitHub Copilot คืออะไร?

AI Pair Programmer ที่ทำงานร่วมกับคุณใน VS Code
ช่วยเขียนโค้ด แนะนำ และ review ให้อัตโนมัติ

Code Completion

เขียนโค้ดให้อัตโนมัติ
จาก context รอบข้าง

Chat

ถามคำถาม อธิบายโค้ด
สร้างโค้ดจาก prompt

Inline Suggestions

แนะนำโค้ดแบบ
real-time ขณะพิมพ์

Prompting Strategies for Copilot

01

Be Specific

บอกภาษา, framework, library ที่ต้องการ
"Create a Next.js API route using Prisma ORM..."

02

Give Context

อธิบาย project structure, data model
"Given a User model with email and password fields..."

03

Ask for Security

ระบุให้ Copilot คิดเรื่อง security
"...with input validation and SQL injection prevention"

04

Iterate & Refine

ถามต่อเพื่อปรับปรุง
"Add rate limiting and error handling to this endpoint"

Security-Focused Copilot Prompts

Input Validation	`Add Zod schema validation to this API route. Reject requests that don't match the schema.`
Auth Middleware	`Create a Next.js middleware that verifies JWT tokens and redirects unauthorized users.`
Security Headers	`Add security headers (CSP, HSTS, X-Frame-Options) to next.config.js using the headers() function.`
Rate Limiting	`Implement rate limiting for the login API route. Max 5 attempts per IP per 15 minutes.`

WORKSHOP

Workshop 3: Secure Feature with Copilot

1. **สร้าง Contact Form**

```
Create a contact form page with name, email, message fields using React Hook Form + Zod validation
```

2. **สร้าง API Route**

```
Create a Next.js API route for the contact form with input sanitization and rate limiting
```

3. **เพิ่ม Security Headers**

```
Add Content-Security-Policy and other security headers to next.config.js
```

4. **Review & Harden**

```
Review this Next.js app for security vulnerabilities and suggest improvements
```

06

Build & Deploy

Deploy เว็บแอปบน GitHub Pages

Next.js Static Export

สำหรับ deploy บน GitHub Pages

next.config.js

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  output: 'export',
  basePath: '/my-secure-app',
  images: {
    unoptimized: true,
  },
  // Security headers
  async headers() {
    return [{
      source: '/(.*)',
      headers: [
        { key: 'X-Frame-Options', value: 'DENY' },
        { key: 'X-Content-Type-Options',
          value: 'nosniff' },
      ]
    }];
  }
};
module.exports = nextConfig;
```

GitHub Actions — Auto Deploy

git push → GitHub Actions → npm run build → Deploy to GH Pages → Live!

Copilot Prompt

```
"Create a GitHub Actions workflow file that:
1. Triggers on push to main branch
2. Installs dependencies and builds the Next.js app
3. Deploys the static export to GitHub Pages
4. Include caching for node_modules"
```

WORKSHOP

Workshop 4: Deploy to GitHub Pages

1. **สร้าง GitHub Repository**

   สร้าง repo ใหม่บน github.com

2. **Push โค้ด**

   ```
   git init && git add . && git commit -m 'init'
   git push -u origin main
   ```

3. **ตั้งค่า GitHub Pages**

   Settings → Pages → Source: GitHub Actions

4. **สร้าง Workflow**

   ใช้ Copilot สร้าง .github/workflows/deploy.yml

5. **Verify & Test**

   เปิด URL → ตรวจสอบ Security Headers → ทดสอบ

Security Checklist

สิ่งที่ต้องตรวจสอบก่อน deploy ทุกครั้ง

- Input validation ทุก endpoint
- Parameterized queries / ORM
- Output encoding (XSS prevention)
- CSRF protection
- Security headers (CSP, HSTS, etc.)
- HTTPS everywhere
- Secrets ไม่อยู่ใน repo
- Dependencies อัพเดท (npm audit)
- Rate limiting on auth endpoints
- Error handling ไม่ leak ข้อมูลภายใน

Key Takeaways

Security ต้อง built-in ตั้งแต่เริ่ม
ไม่ใช่เพิ่มทีหลัง

AI (Copilot) ช่วยเร่งงาน
แต่ต้อง review ผลลัพธ์เสมอ

Next.js มี security features
ในตัว — ใช้ให้เป็นประโยชน์

CI/CD ช่วยให้ deploy
อย่างปลอดภัยและสม่ำเสมอ

Q & A

ถาม-ตอบ และแลกเปลี่ยนความคิดเห็น